Pandas-Zuweisung für Viterbi-Matrizen

https://gemini.google.com/app/70a309c9af023195

User prompt: Pandas: df2 = viterbi = word Weithin reicht die Sicht in die Manila-Bucht , die wegen ihres \ name $(NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN $, NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN $. NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN -- NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN ADJA NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN ... ... QL NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN S NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN VP NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN VZ NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN VROOT NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN word Sonnenuntergangs weltberühmt ist . name $(NaN NaN NaN NaN $, NaN NaN NaN NaN $. NaN NaN NaN NaN -- NaN NaN NaN NaN ADJA NaN NaN NaN NaN ... ... QL NaN NaN NaN NaN S NaN NaN NaN NaN VP NaN NaN NaN NaN VZ NaN NaN NaN NaN VROOT NaN NaN NaN NaN df1 = init_tag_prob = word pos word_pos_count pos_count word_count word_pos_prob 3 die ART 20127 92486 23213 0.867057 27 die PRELS 2936 5888 23213 0.126481 483 die PDS 144 3086 23213 0.006203 13948 die PIS 5 4561 23213 0.000215 50948 die PWAV 1 1641 23213 0.000043 Die Werte in Spalte word_pos_prob sollen in die korresbondierenden Zellen in viterbi eingetragen werden. Der Index "name" in df2 entspricht der Spalte "pos" in df1. Für jede Spalte in df2 wird eine eigene df1 erstellt und muss ebenfalls in df2 übertragen werden. Wie lautet die Pandas-Anweisung? Wie lautet die übersichtliche Anweisung?

Response: Hier sind die effizientesten Pandas-Anweisungen, um dieses Mapping durchzuführen. Pandas erledigt die korrekte Zuordnung der Zeilen (POS-Tags) durch das sogenannte **Index-Alignment** vollautomatisch.

Die übersichtlichste Anweisung (Index-Alignment)

Wenn Sie `df1` für ein bestimmtes Wort (z.B. "die") generiert haben, können Sie die Werte direkt übergeben, indem Sie die Spalte `pos` vorübergehend zum Index machen.

```
# Das aktuelle Wort ermitteln (hier: 'die')
wort = df1['word'].iloc[0]

# Die übersichtliche Anweisung:
viterbi[wort] = df1.set_index('pos')['word_pos_prob']
```

Warum das so gut funktioniert: Durch `.set_index('pos')` weiß Pandas, welcher Wert zu welchem Tag gehört (z. B. `ART` -> `0.867057`). Pandas gleicht diesen temporären Index nun automatisch mit dem Index `name` in `df2` (viterbi) ab. Alle anderen Zellen in dieser Spalte bleiben bzw. werden automatisch `NaN`.

Wichtiger Hinweis für Viterbi (Doppelte Wörter im Satz)

Ihr Testsatz lautet: *"Weithin reicht **die** Sicht in **die** Manila-Bucht, **die** wegen..."* Das Wort **die** kommt mehrfach vor, was bedeutet, dass Ihr DataFrame `viterbi` **mehrere gleichnamige Spalten** hat.

Wenn Sie `viterbi['die'] = ...` aufrufen, versucht Pandas alle Spalten namens "die" gleichzeitig zu überschreiben. Das kann je nach Pandas-Version zu einem `ValueError` führen (da eine 1D-Serie auf eine 2D-Matrix zugewiesen wird).

Die sichere Anweisung für Ihre Iteration: Da Sie den Viterbi-Algorithmus vermutlich in einer Schleife Wort für Wort von links nach rechts aufbauen, sollten Sie über die Spalten-Position (Index) gehen, nicht über den Spaltennamen. Nutzen Sie dafür `.iloc`:

```
# Angenommen, Sie iterieren über die Spalten von df2:
for i, wort in enumerate(viterbi.columns):

    # ... Ihr Code, der df1 für das aktuelle 'wort' an Position 'i' erstellt ...

    # Positionsbasierte Zuweisung für die i-te Spalte
    viterbi.iloc[:, i] = df1.set_index('pos')['word_pos_prob']
```

Mit `.iloc[:, i]` greifen Sie exakt auf die Spalte an der aktuellen Position der Schleife zu. Das Index-Alignment funktioniert hier exakt genauso elegant, Sie vermeiden aber jegliche Fehler durch doppelte Wörter im Satz.

User prompt: wort = df1['word'].iloc[0] viterbi[wort] = df1.set_index('pos')['word_pos_prob'] Läuft einmal wie erwartet durch. Mit einem neuen Wort in "word" läuft es ohne Fehlermeldung durch, aberes wird kein neuer Wert in viterbi eingetragen, weder in bestehende Zeile in die entsprechende Spalte, noch in eine neue Zeile, falls es einen passenden Wert in 'pos' noch nicht geben sollte. Wo liegt der Fehler? Wie kann er behoben werden?

Response: Der Fehler liegt in zwei spezifischen Verhaltensweisen von Pandas, die bei solchen Zuweisungen oft für Verwirrung sorgen.

Wo liegt der Fehler?

1. **Pandas erzeugt bei `=` keine neuen Zeilen:** Wenn Sie `viterbi[wort] = Series` ausführen, gleicht Pandas die Daten strikt anhand des bestehenden Index (Ihre POS-Tags) ab. Wenn `df1` ein neues POS-Tag enthält (z. B. `VVFIN` für "reicht"), das in `viterbi.index` noch nicht existiert, wird dieser Wert **stillschweigend weggeworfen**. Pandas erweitert den Index bei einer Zuweisung über `=` niemals automatisch.
2. **Komplette Spalten-Überschreibung:** Wenn das neue Wort ausschließlich Tags besitzt, die noch nicht im Index stehen, verpuffen alle Werte. Die gesamte Spalte wird dann einfach mit `NaN` gefüllt (oder bleibt unberührt, falls sie leer war), und der Code läuft ohne Fehlermeldung weiter.

Wie kann er behoben werden?

Sie müssen Pandas explizit anweisen, fehlende POS-Tags als neue Zeilen anzulegen, *bevor* Sie die Daten zuweisen.

Hier ist die robuste und übersichtliche Lösung, die Sie ideal in Ihre Viterbi-Schleife einbauen können:

```
# 1. Die neuen Werte als Series vorbereiten (Index = 'pos')
neue_werte = df1.set_index('pos')['word_pos_prob']

# 2. Prüfen, ob es POS-Tags gibt, die noch nicht in viterbi als Zeile existieren
fehlende_tags = neue_werte.index.difference(viterbi.index)

# 3. Fehlende Tags als neue leere Zeilen anlegen
for tag in fehlende_tags:
    viterbi.loc[tag] = None  # Erzeugt eine neue Zeile, komplett gefüllt mit NaN

# 4. Werte eintragen
# (Wenn Sie aktuell nur einzelne Wörter manuell testen:)
viterbi[wort] = neue_werte
```

⚠️ Wichtige Vorwarnung für Ihren Viterbi-Algorithmus

Da Sie einen Satz verarbeiten (*"Weithin reicht **die** Sicht in **die** Manila-Bucht, **die** wegen..."*), kommt das Wort **"die"** mehrfach vor.

Wenn Sie `viterbi[wort] = neue_werte` nutzen und `wort == "die"` ist, wird Pandas **alle drei Spalten namens "die" gleichzeitig überschreiben**. Das würde Ihren Viterbi-Algorithmus sofort zerstören, da die Positionen im Satz unterschiedliche Wahrscheinlichkeiten (Pfade) berechnen müssen.

Die Endlösung für Ihre Iteration: Greifen Sie bei der Zuweisung nicht über den String (`wort`), sondern über die **Spaltenposition (Index `i`)** zu.

```
# Angenommen, Sie iterieren so über den Satz:
# for i, wort in enumerate(viterbi.columns):

    # ... Ihr Code, der df1 für das aktuelle Wort erstellt ...

    neue_werte = df1.set_index('pos')['word_pos_prob']

    fehlende_tags = neue_werte.index.difference(viterbi.index)
    for tag in fehlende_tags:
        viterbi.loc[tag] = None

    # SICHERE ZUWEISUNG: Überschreibt nur die exakte Spalte an Position i
    viterbi.iloc[:, i] = neue_werte
```

Mit `.iloc[:, i] = neue_werte` gleicht Pandas die POS-Tags (Zeilen) weiterhin perfekt ab, trägt sie aber garantiert nur in die gerade aktuelle Spalte des Satzes ein.

User prompt: Hier mein Code: for i, wort in enumerate(viterbi.columns): # ... Ihr Code, der df1 für das aktuelle Wort erstellt ... init_tag_prob = train_word_pos_count[train_word_pos_count["word"] == "Weithin"] neue_werte =

init_tag_prob.set_index('pos')['word_pos_prob'] fehlende_tags = neue_werte.index.difference(viterbi.index) for tag in fehlende_tags: viterbi.loc[tag] = None # SICHERE ZUWEISUNG: Überschreibt nur die exakte Spalte an Position i viterbi.iloc[:, i] = neue_werte Fehlermeldung: ---> 14 viterbi.iloc[:, i] = neue_werte ValueError: Must have equal len keys and value when setting with an iterable Was ist die Fehlerursache? Wie beheben?

Response: Die Fehlerursache liegt im Unterschied zwischen **Label-basierter** Zuweisung (mit Namen) und **Positions-basierter** Zuweisung (mit `.iloc`).

Die Fehlerursache

Wenn Sie `.iloc[:, i]` nutzen, um eine komplette Spalte zu überschreiben, verlangt Pandas zwingend, dass die Daten auf der rechten Seite (`neue_werte`) **exakt dieselbe Länge** (Anzahl der Elemente) haben wie die Spalte auf der linken Seite.

Ihr DataFrame `viterbi` hat bereits viele Zeilen (z. B. 50 verschiedene POS-Tags). Das gefilterte Wort (z. B. "Weithin") hat aber vielleicht nur 1 oder 2 mögliche POS-Tags. Pandas versucht nun, eine Series der Länge 2 in eine Spalte der Länge 50 zu quetschen. Bei der Zuweisung über den Spaltennamen (`viterbi[wort] = ...`) hätte Pandas das über das Index-Alignment intelligent selbst geregelt, aber `.iloc` ist dumm und stur positionsbasiert – daher der `ValueError`.

Die Lösung: `.reindex()`

Wir müssen `neue_werte` einfach kurz auf die exakt gleiche Länge und Reihenfolge wie den `viterbi`-Index aufblasen, bevor wir es `.iloc` übergeben. Pandas füllt die restlichen Tags dann automatisch mit `NaN` auf. Das geht ganz elegant mit `.reindex()`.

Hier ist Ihr korrigierter Code (inklusive eines kleinen Hinweises zu Ihrem hartcodierten `"Weithin"`):

```
for i, wort in enumerate(viterbi.columns):

    # Hinweis: Für den produktiven Lauf sollten Sie hier "wort" statt "Weithin" nutzen!
    init_tag_prob = train_word_pos_count[train_word_pos_count["word"] == wort]

    neue_werte = init_tag_prob.set_index('pos')['word_pos_prob']

    # 1. Prüfen und Hinzufügen noch völlig unbekannter Tags
    fehlende_tags = neue_werte.index.difference(viterbi.index)
    for tag in fehlende_tags:
        viterbi.loc[tag] = None

    # 2. LÖSUNG: Passen Sie die Länge der Series exakt an den Viterbi-Index an.
    # Alle Tags, die das Wort nicht hat, bekommen dabei automatisch ein NaN.
    neue_werte_angepasst = neue_werte.reindex(viterbi.index)

    # 3. Jetzt klappt die Zuweisung, da beide Seiten exakt gleich lang sind.
    viterbi.iloc[:, i] = neue_werte_angepasst
```

Was `.reindex(viterbi.index)` unter der Haube macht: Es nimmt Ihre kleine Series (z. B. `ADV: 0.05`) und gleicht sie mit allen Zeilennamen in `viterbi` ab. Das Ergebnis ist eine Series in voller Länge (z. B. `$(: NaN`, `$,: NaN`, ..., `ADV: 0.05`, ...). Dadurch sind linke und rechte Seite der Gleichung gleich lang und `.iloc` ist zufrieden.

User prompt: Was würde sich an der Lösung ändern, wenn viterbi mit allen mögliche POS-Tags zu beginn initiert werden würde? Würde das den Merge vereinfachen?

Response: Ja, das vereinfacht den Code erheblich und verbessert die Performance spürbar.

Wenn der Index von `viterbi` bereits zu Beginn alle möglichen POS-Tags (z. B. den vollständigen STTS-Tagset mit 54 Tags) enthält, ändert sich Folgendes:

1. **Die Schleife schrumpft massiv:** Der gesamte Block zum Suchen fehlender Tags (`fehlende_tags`) und das dynamische Anlegen neuer Zeilen (`viterbi.loc[tag] = None`) entfällt ersatzlos.
2. **Deutlicher Leistungsgewinn:** Das schrittweise Vergrößern eines DataFrames in einer Schleife erzwingt ständige Speicher-Reallokationen und ist in Pandas sehr langsam. Ein von Anfang an fixer Index ist wesentlich effizienter.

Der vereinfachte Code

```
# viterbi ist bereits mit allen POS-Tags als Index initialisiert
for i, wort in enumerate(viterbi.columns):
    init_tag_prob = train_word_pos_count[train_word_pos_count["word"] == wort]
    neue_werte = init_tag_prob.set_index('pos')['word_pos_prob']

    # Direktes Mapping: reindex passt die Werte an den festen viterbi-Index an
    viterbi.iloc[:, i] = neue_werte.reindex(viterbi.index)
```

Optionale Optimierung für Nullen (`fill_value=0.0`)

Wenn unbeobachtete Wort-Tag-Kombinationen im Viterbi-Algorithmus nicht als `NaN`, sondern direkt als Wahrscheinlichkeit `0.0` repräsentiert werden sollen, lässt sich das direkt beim `.reindex()` festlegen:

```
viterbi.iloc[:, i] = neue_werte.reindex(viterbi.index, fill_value=0.0)
```
